

3.3 — Apply Path-Specific Rules for Conditional Convention Loading

Concept

Claude Code loads rules from `.claude/rules/` and applies them globally by default. Path-specific rules refine this behavior: a rule file can declare a `paths` field in its YAML frontmatter, and Claude will activate that rule only when the files being edited match the declared patterns. This is conditional convention loading — rules become active or inactive depending on the workspace context.

Why It Matters

A large codebase has heterogeneous concerns. React component standards should not bleed into database migration scripts. API authentication conventions should not apply to unit test helpers. Without path specificity, every rule is always in play, creating noise, contradictions, and cognitive overhead for the agent.

Path-specific rules give you:

- **Precision** — conventions apply only where they are relevant
- **Conflict avoidance** — rules for different layers of the stack cannot collide
- **Maintainability** — each rule file owns a single concern, scoped to a single area

YAML Frontmatter `paths` Field

A rule file is a Markdown file with an optional YAML frontmatter block at the top. The `paths` field accepts a list of glob patterns. Claude evaluates the patterns against the paths of files currently being edited or referenced in the conversation.

```
---
paths:
  - "src/components/**/*.tsx"
  - "src/components/**/*.ts"
---
```

If any active file matches at least one of the listed patterns, the rule is loaded. If no file matches, the rule is silently skipped.

Glob Pattern Syntax

Glob patterns follow standard Unix conventions with a few common forms:

Pattern	Matches
<code>src/**/*.ts</code>	All <code>.ts</code> files anywhere under <code>src/</code>
<code>src/api/**/*.ts</code>	Only <code>.ts</code> files under <code>src/api/</code>
<code>**/__tests__/**</code>	Any <code>__tests__</code> directory at any depth
<code>*.config.{js,ts}</code>	Config files in the project root
<code>src/components/**</code>	All files (any extension) under <code>src/components/</code>

Use `**` to match any number of path segments, `*` to match within a single segment, and `{a,b}` to match alternatives.

Code Example

Rule: React Component Standards

```

---
description: React component conventions for the component library
paths:
  - "src/components/**/*.tsx"
  - "src/components/**/*.stories.tsx"
---

# React Component Standards

- Export components as named exports, never default exports.
- Colocate the component, its types, and its styles in a single directory.
- Props interfaces must be prefixed with the component name: `ButtonProps`, not `Props`.
- Never access `window` or `document` directly; use a custom hook that guards SSR.

```

Rule: API Route Patterns

```

---
description: Express route and controller conventions
paths:
  - "src/api/routes/**/*.ts"
  - "src/api/controllers/**/*.ts"
---

# API Route Patterns

- Validate all request bodies with Zod before touching business logic.
- Controllers must not import from other controllers; only from services.
- Return consistent envelope: `{ success, data, error }`.
- All routes must be covered by an integration test in `src/api/__tests__`.

```

Rule: Testing Conventions

```

---
description: Test authoring standards for unit and integration tests
paths:
  - "**/__tests__/**/*.ts"
  - "**/*.test.ts"
  - "**/*.spec.ts"
---

# Testing Conventions

- One `describe` block per module under test.
- Use `vi.mock()` at the module level, never inside `beforeEach`.
- Assert on behavior, not implementation details.
- Every test must clean up side effects in `afterEach`.

```

With these three files in `.claude/rules/`, a session editing `src/components/Button/Button.tsx` loads only the component rule. A session editing `src/api/routes/users.ts` loads only the API rule. A session editing `src/api/__tests__/users.test.ts` loads both the API rule and the testing rule, because the file matches patterns in each.

Common Mistakes

Overly broad patterns. Using `**/*.ts` in a rule intended for API controllers will activate the rule everywhere TypeScript is written. Tighten the path prefix to the specific subdirectory.

Conflicting rules for the same path. Two rules that both match `src/api/**/*.ts` and give contradictory instructions create ambiguity. Scope rules by concern and ensure no two rules make opposing claims about the same files.

Forgetting extension variants. A component in `src/components/` might be `.tsx`, `.ts`, `.stories.tsx`, or `.test.tsx`. If your pattern only covers `*.tsx`, the rule silently drops for the `.ts` files. List all relevant extensions or use `src/components/**` without an extension filter when the rule applies regardless of file type.

Paths relative to the wrong root. Patterns are matched against paths relative to the project root, not the `.claude/` directory. Write `src/components/**`, not `../..src/components/**`.

Missing a rule file entirely. When a convention is not enforced by any rule, Claude falls back to general behavior. If your team has strong opinions about a particular area of the codebase, make that a rule file with an explicit `paths` field.

References

- Claude Code documentation: `.claude/rules/` directory and frontmatter schema
- Topic 3.1 — Structuring `.claude/rules/` for project-wide conventions
- Topic 3.2 — Writing effective rule descriptions and instruction prose
- Topic 3.4 — Resolving rule precedence and merge order

